



Facultad de Ingeniería - Udelar

Laboratorio

Cálculo de la distancia entre secuencias de ADN

Belén Olivera

Nicolás Pereira

Manuel Roca

1. Introducción

El objetivo de este laboratorio es implementar una versión eficiente sobre GPU para el cálculo de la matriz de distancias euclídeas entre individuos representados como secuencias de SNPs. Dado que la cantidad de individuos y SNPs puede ser muy grande, el problema a tratar consiste en aprovechar el paralelismo masivo y la jerarquía de memorias de la GPU para obtener un buen desempeño. Siguiendo la estrategia propuesta en la letra, el cálculo de la distancia euclídea se plantea de la siguiente forma:

$$D^2 = \tilde{N} + \tilde{N}^T - 2XX^T \quad (1)$$

donde $\tilde{N}$ es una matriz cuyas filas contienen repetida la norma cuadrada de cada individuo. Esto permite expresar la mayor parte del trabajo como un producto de matrices XX^T .

Teniendo en cuenta esta formulación del problema, desarrollamos distintas implementaciones que incorporan distintas técnicas, centrandonos en la multiplicación XX^T , con el fin de analizar su performance y compararlas. A la vez, comparamos el desempeño con una implementación que usa la operación cublasSyrk de la biblioteca cuBlas.

2. Diseño de la solución

Todas las implementaciones desarrolladas siguen una misma estructura general, dividiendo el cálculo de la matriz de distancias en cuatro etapas, cada una implementada en un kernel independiente.

En primer lugar, se calcula la norma cuadrada de cada fila de la matriz de entrada, obteniendo el vector de normas. A continuación, se computa el producto matricial XX^T , que es la etapa de mayor costo computacional y constituye el núcleo de todas las implementaciones.

Una vez disponibles ambos resultados, se construye la matriz de distancias al cuadrado aplicando la expresión 1, que combina el producto matricial con las normas previamente calculadas. Por último, se obtiene la matriz de distancias euclídeas aplicando la raíz cuadrada a cada elemento.

Las distintas versiones presentadas en este informe mantienen esta organización general y difieren únicamente en las estrategias empleadas para optimizar las etapas. La mayor parte del trabajo se concentró en optimizar el kernel de multiplicación XX^T , que insume la mayor parte del trabajo de la operación completa.

3. Versiones implementadas

A lo largo del trabajo desarrollamos más versiones y variantes preliminares de las que se presentan aquí, incluyendo aproximaciones intermedias que luego descartamos o fuimos refinando. Por claridad y por el límite de extensión del informe, en esta sección documentamos únicamente las versiones que consideramos más relevantes para ilustrar las decisiones de diseño y la evolución de las optimizaciones sobre el kernel de multiplicación XX^T .

3.1. Baseline CUDA

En esta primera versión, que utilizamos como baseline, implementamos el producto XX^T utilizando la técnica de tiling con memoria compartida. El objetivo principal fue reducir las lecturas

redundantes desde memoria global mediante la reutilización de datos desde memoria compartida, y también aprovechar la simetría de la matriz resultante.

Utilizamos bloques de tamaño 32×32 , donde cada uno calcula un tile del mismo tamaño para la matriz resultado. Elegimos este tamaño de bloque pues, por la letra del laboratorio, las matrices X son de dimensiones múltiplo de 32, lo que simplifica el problema y evita calcular tiles incompletos.

En cada iteración sobre la dimensión m de la matriz $X_{n \times m}$, los hilos del bloque cargan dos tiles, tile_A y tile_B. Cada tile contiene un bloque de 32×32 elementos correspondiente a un subconjunto de filas de X y a un bloque de 32 columnas consecutivas de la dimensión m , que se cargan de forma coalesced. Ambas tiles son de dimensión 32×33 , donde se le agregó padding para evitar conflictos de banco al acceder a la memoria compartida.

Una vez que se cargaron las tiles, se hace `__syncthreads()` para asegurarnos de que todos los hilos han terminado de cargar los elementos correspondientes. Luego, a cada hilo le corresponde calcular el producto interno entre un par de filas de la matriz, por lo que lee directamente desde tile_A y tile_B, calcula el aporte al producto interno que logra con estos elementos, y acumula este valor en una variable. De esta forma, cada dato leído desde memoria global se reutiliza por múltiples hilos del bloque, evitando volver a cargarlo y aprovechando la menor latencia de la memoria compartida.

Antes de hacer la siguiente iteración se ejecuta otro `__syncthreads()` para asegurarnos que todos los hilos hayan terminado de utilizar el contenido actual de la memoria compartida antes de sobrescribirlo con la siguiente tile.

Se repite el proceso para todos los tiles necesarios hasta recorrer las m columnas de la matriz. Finalmente, una vez acumuladas todos los productos parciales, cada hilo almacena en memoria global el elemento correspondiente de la matriz resultante XX^T .

Además, como la matriz resultante XX^T es simétrica, se calcula solamente la mitad triangular superior. Los bloques correspondientes a la mitad inferior retornan inmediatamente al principio del kernel con la condición:

```
if (blockIdx.x < blockIdx.y) {
    return;
}
```

Esto nos permite ahorrar el cálculo de aproximadamente la mitad de los productos. Si bien es cierto que en los bloques diagonales hay hilos que calculan elementos de la mitad triangular inferior, decidimos retornar a nivel bloque en vez de a nivel hilos. Esto es para evitar divergencias a nivel de warp, a cambio se realizan algunos cálculos redundantes en los bloques diagonales, pero consideramos que este costo es menor para la GPU. En los otros kernels que completan la operación de distancia euclídea se accede de forma “espejada” a la matriz XX^T , reutilizando los valores de la mitad triangular superior para obtener los elementos correspondientes de la mitad inferior.

3.2. Empaquetado de bits (bit packing)

En esta segunda versión se buscó aprovechar que los valores posibles de cada SNP son 0, 1 y 2. Como estos valores pueden representarse utilizando solamente dos bits (00, 01 y 10), se implementó una representación compacta de la matriz de entrada X , almacenando varios SNPs dentro de un mismo entero.

El procedimiento general del kernel XX^T sigue siendo prácticamente igual al de la versión

anterior. Se continúa utilizando tiling con memoria compartida, accesos coalesced y el aprovechamiento de la simetría de la matriz. La principal diferencia es que ahora cada posición de memoria compartida contiene varios SNPs empaquetados. Por ejemplo, mientras que en la versión base una tile de 32×32 contiene únicamente 32 SNPs por fila, en esta versión cada entero almacena 16 SNPs, permitiendo representar hasta 512 SNPs por fila utilizando exactamente la misma cantidad de memoria compartida.

Una vez que se cargaron los datos en memoria compartida, cada hilo sigue siendo responsable del producto interno entre un par de filas. Sin embargo, en lugar de procesar un SNP por vez, ahora opera sobre grupos de 16 SNPs empaquetados dentro de un mismo entero de 32 bits. Dado que cada SNP se representa con 2 bits (valor = 2·bit_alto + bit_bajo, con valores posibles 0, 1 o 2), el producto entre dos SNPs a y b puede descomponerse en cuatro términos según la combinación de sus bits altos, que llamamos a_1 y b_1 , y los bajos, que llamamos a_0 y b_0 , de la siguiente forma:

$$a \cdot b = (2a_1 + a_0)(2b_1 + b_0) = 4a_1b_1 + 2a_1b_0 + 2a_0b_1 + a_0b_0$$

Y como a_1, a_0, b_1, b_0 son bits individuales (0 o 1), cada producto $a_i * b_j$ coincide exactamente con el AND bit a bit $a_i \& b_j$. Esto permite reescribir la expresión como:

$$4(a_1 \& b_1) + 2(a_1 \& b_0) + 2(a_0 \& b_1) + (a_0 \& b_0)$$

Aplicando máscaras (0x55555555) y un shift de 1 bit, se separan los bits bajos (a_0, b_0) y los bits altos (a_1, b_1) de los 16 SNPs empaquetados en cada entero de 32 bits. Como esta operación se aplica en paralelo sobre las 16 posiciones simultáneamente, la instrucción `_popc` permite contar en una sola operación [2] en cuántas de esas 16 posiciones el AND dio 1, obteniendo así la suma de los 16 productos parciales de una sola vez:

$$4 \cdot _popc(a_1 \& b_1) + 2 \cdot _popc(a_1 \& b_0) + 2 \cdot _popc(a_0 \& b_1) + _popc(a_0 \& b_0)$$

La motivación de esta representación es doble. Por un lado, al almacenar varios SNPs dentro de un mismo entero, una misma tile de memoria compartida representa muchos más SNPs sin aumentar su tamaño, y esto reduce la cantidad de iteraciones sobre la dimensión m . Además, la matriz empaquetada ocupa menos memoria, tanto en la transferencia desde CPU a GPU como en los accesos a memoria global. Por otro lado, la representación permite calcular cada producto interno con muy pocas operaciones: una primera aproximación que probamos desempaquetaba cada grupo de 16 SNPs para procesarlos de a uno, lo que requería desplazamientos, máscaras y multiplicaciones por elemento, y la descartamos. En su lugar, las operaciones bit a bit junto con la instrucción de hardware `_popc` permiten obtener el producto de los 16 SNPs empaquetados en un número mucho menor de instrucciones.

3.3. Tensor cores

En esta tercer versión se buscó aprovechar los Tensor Cores disponibles en las GPU NVIDIA. Para esto se utilizó la API WMMA, que permite acceder a las instrucciones de multiplicación matricial implementadas por hardware.

Los Tensor Cores operan sobre matrices pequeñas de tamaño fijo (en este caso usamos $16 \times 16 \times 16$), utilizando operandos en precisión half y acumulando el resultado en precisión float. Por este motivo, antes de ejecutar el producto matricial se incorpora un kernel adicional que convierte la matriz de entrada de `uint8_t` a half. Esta conversión se realiza una única vez y su costo resulta despreciable frente al tiempo total del cálculo de XX^T .

En las versiones anteriores cada hilo calcula un único elemento de la matriz resultado mediante un producto interno, pero en esta implementación el trabajo se organiza a nivel de warp. Cada warp calcula una subtile de 16×16 , ejecutado directamente por un Tensor Core usando la instrucción `wmma::mma_sync` para multiplicación y acumulación.

Para mejorar la reutilización de los datos se mantiene el uso de tiling con memoria compartida. Cada bloque está compuesto por 128 hilos, equivalentes a cuatro warps. En conjunto, estos cuatro warps calculan un bloque de salida de 32×32 , formado por cuatro subtiles de 16×16 .

En cada iteración sobre la dimensión m , los 128 hilos del bloque cargan desde memoria global dos tiles de tamaño 32×16 en memoria compartida. Una vez cargados los datos, cada warp toma únicamente el subtile de 16×16 que necesita utilizando `wmma::load_matrix_sync`, y se ejecuta la operación `wmma::mma_sync(c_frag, a_frag, b_frag, c_frag)` que hace la multiplicación de ambos subtiles y acumula el resultado en el fragmento `c_frag`. Este proceso se repite para todos los bloques de 16 columnas de la matriz, acumulando sucesivamente los productos parciales hasta obtener el bloque completo de la matriz XX^T .

Al terminar el recorrido sobre toda la dimensión m de la matriz, cada warp almacena su subtile de 16×16 en memoria global usando `wmma::store_matrix_sync`.

De esta forma, el cálculo de los productos internos deja de hacerse mediante multiplicaciones y sumas ejecutadas por los CUDA Cores, y se pasa a utilizar unidades de hardware especializadas en multiplicación de matrices. Esto permite incrementar significativamente el rendimiento del kernel XX^T , especialmente para matrices de gran tamaño.

3.4. Empaquetado de bit + Tensor Cores

Esta versión combina las dos ideas centrales de las versiones anteriores: el empaquetado a nivel de bits de la sección de empaquetado de bits y el uso de Tensor Cores de la sección anterior.

Como ya vimos, cada SNP se escribe como $x = 2a_1 + a_0$, y el producto interno de dos individuos se descompone en cuatro productos entre sus bits altos y bajos. La novedad acá es que, en lugar de empaquetar los dos bits juntos, separamos el bit alto y el bit bajo en dos matrices independientes (X_1 y X_0), de modo que el producto interno queda como suma de cuatro AND-popcount entre planos:

$$\langle x_i, x_j \rangle = 4 \langle a_{1i}, a_{1j} \rangle + 2 \langle a_{1i}, a_{0j} \rangle + 2 \langle a_{0i}, a_{1j} \rangle + \langle a_{0i}, a_{0j} \rangle \quad (2)$$

Cada $\langle u, v \rangle$ cuenta las posiciones donde ambos bits valen 1, que es exactamente la operación que un Tensor Core binario resuelve de forma masiva.

El principal desafío fue de compatibilidad de hardware. La operación AND-popcount por hardware (`bmmaBitOpAND`) solo está disponible en dispositivos con compute capability 8.0 o superior [3], mientras que la GPU utilizada (NVIDIA GeForce RTX 2080 Ti) es de compute capability 7.5. Por este motivo solo disponemos de la variante XOR-popcount (`bmmaBitOpXOR`). Para solucionar esto usamos una identidad exacta que reconstruye el AND a partir del XOR:

$$\langle u, v \rangle = \frac{|u| + |v| - \text{XOR-popc}(u, v)}{2} \quad (3)$$

donde $|u|$ es el popcount total de la fila u . Lo conveniente es que estos popcounts por fila de cada plano ($S_1[i]$ y $S_0[i]$) son los mismos que de todas formas necesitábamos para las normas. Por lo tanto, la corrección XOR→AND se aplica una única vez en el post-proceso y el kernel WMMA queda idéntico salvo por reemplazar AND por XOR. Definiendo el acumulado ponderado de los cuatro XOR-popcounts como

$$W_{ij} = 4 \text{XOR}(a_{1i}, a_{1j}) + 2 \text{XOR}(a_{1i}, a_{0j}) + 2 \text{XOR}(a_{0i}, a_{1j}) + \text{XOR}(a_{0i}, a_{0j}) \quad (4)$$

se obtiene el producto y la norma en aritmética entera, y por lo tanto exacta:

$$2 \langle x_i, x_j \rangle = 6 (S_{1i} + S_{1j}) + 3 (S_{0i} + S_{0j}) - W_{ij} \quad (5)$$

$$\|x_i\|^2 = 4 S_{1i} + S_{0i} \quad (6)$$

La organización del kernel sigue el mismo esquema a nivel de warp y con tiling en memoria compartida de la versión con Tensor Cores. La diferencia es que los Tensor Cores binarios operan sobre tiles de $8 \times 8 \times 128$, es decir, cada operación consume un tramo de $K = 128$ bits (4 `uint32_t`) de la dimensión m , y ahora se stagean las 32 filas de A y de B en ambos planos a la vez:

```
__shared__ uint32_t As1[32][4], As0[32][4]; // plano alto y bajo de A
__shared__ uint32_t Bs1[32][4], Bs0[32][4]; // plano alto y bajo de B
```

Cada warp ejecuta los cuatro productos binarios con `wmma::bmma_sync` (usando `bmmaBitOpXOR` y `bmmaAccumulateOpPOPC`), acumulando cada uno en su fragmento correspondiente (c_{11} , c_{10} , c_{01} , c_{00}). Al terminar el recorrido sobre m , se combinan los cuatro acumuladores en el peso $W = 4c_{11} + 2c_{10} + 2c_{01} + c_{00}$ y se almacena en memoria global con `wmma::store_matrix_sync`. Como en las versiones anteriores, se sigue aprovechando la simetría de la matriz para calcular únicamente el triángulo superior. Finalmente, el kernel de post-proceso reconstruye la matriz XX^T a partir de W y los popcounts S_1, S_0 , aplica la corrección XOR→AND y arma la matriz de distancias, todo en aritmética entera hasta el último paso.

4. Comparación de resultados

4.1. Tiempos de ejecución multiplicación

Para evaluar el rendimiento tomamos como referencia la versión `Cublas`, que calcula el producto XX^T con `cublasSsyrrk` (FP32) y resuelve el resto del pipeline con kernels simples. Incluimos además la versión `Baseline CUDA` como la implementación propia más simple sobre CUDA cores.

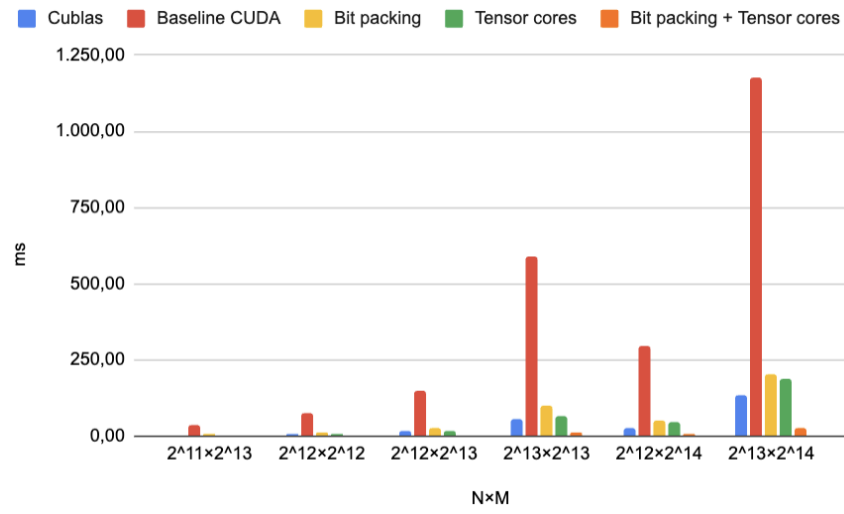
La GPU utilizada fue una NVIDIA GeForce RTX 2080 Ti y todas las compilaciones fueron de la forma `nvcc -O3 -arch=sm_75 version.cu -o version`.

El *speedup* se calcula como `tiempo_cublas/version` y el error relativo como $\|D - D_{\text{cuBLAS}}\| / \|D_{\text{cuBLAS}}\|$. En todos los casos se constató que el error relativo fue exactamente 0.

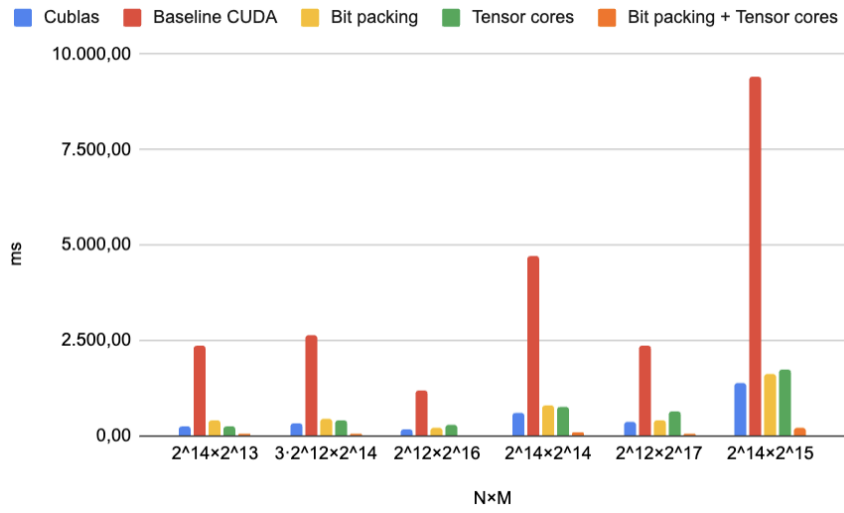
N	M	Cublas	Baseline CUDA	Bit packing	Tensor cores	Bit packing + Tensor cores
2^{11}	2^{13}	3.683	38.035	6.605	4.254	0.811
2^{12}	2^{13}	19.198	148.488	25.515	16.242	3.170
2^{13}	2^{13}	54.701	589.473	100.860	63.611	12.800
2^{14}	2^{13}	252.867	2351.741	402.249	252.982	50.503
2^{12}	2^{12}	7.060	74.455	12.806	8.218	1.571
2^{12}	2^{14}	25.744	296.752	50.932	46.173	6.429
2^{12}	2^{16}	169.915	1187.711	202.723	278.116	25.131
2^{12}	2^{17}	353.565	2375.510	405.298	655.905	50.081
2^{13}	2^{14}	135.478	1178.531	201.398	187.402	25.202
$3 \cdot 2^{12}$	2^{14}	338.470	2646.750	452.687	423.740	56.556
2^{14}	2^{14}	618.467	4701.673	803.573	752.668	100.366
2^{14}	2^{15}	1376.264	9407.162	1606.617	1733.688	198.032

Cuadro 1: Tiempos de ejecución del kernel XX^T en ms para las distintas implementaciones, medidos con `cudaEvent`.

Incluimos en la Figura 1 gráficos para poder visualizar mejor los resultados.



(a) Matrices pequeñas.



(b) Matrices grandes.

Figura 1: Comparación de los tiempos de ejecución para distintos tamaños de entrada.

4.2. Tiempos de ejecución pipeline completo

N	M	Cublas	Baseline CUDA	Bit packing	Tensor cores	Bit packing + Tensor cores
2^{11}	2^{13}	10.840 (1.00×)	45.127 (0.24×)	10.260 (1.06×)	11.413 (0.95×)	4.549 (2.38×)
2^{12}	2^{13}	37.119 (1.00×)	166.415 (0.22×)	38.050 (0.98×)	34.197 (1.09×)	15.789 (2.35×)
2^{13}	2^{13}	110.691 (1.00×)	645.539 (0.17×)	147.595 (0.75×)	119.694 (0.92×)	59.740 (1.85×)
2^{14}	2^{13}	452.242 (1.00×)	2551.695 (0.18×)	583.389 (0.78×)	452.551 (1.00×)	232.135 (1.95×)
2^{12}	2^{12}	21.588 (1.00×)	88.991 (0.24×)	24.594 (0.88×)	22.748 (0.95×)	13.456 (1.60×)
2^{12}	2^{14}	50.497 (1.00×)	321.517 (0.16×)	64.888 (0.78×)	71.171 (0.71×)	20.502 (2.46×)
2^{12}	2^{16}	235.508 (1.00×)	1253.343 (0.19×)	225.192 (1.05×)	343.731 (0.69×)	47.657 (4.94×)
2^{12}	2^{17}	473.555 (1.00×)	2495.688 (0.19×)	439.109 (1.08×)	776.053 (0.61×)	83.953 (5.64×)
2^{13}	2^{14}	203.585 (1.00×)	1246.645 (0.16×)	250.837 (0.81×)	255.523 (0.80×)	74.846 (2.72×)
$3 \cdot 2^{12}$	2^{14}	474.820 (1.00×)	2783.164 (0.17×)	560.212 (0.85×)	560.271 (0.85×)	164.002 (2.90×)
2^{14}	2^{14}	841.900 (1.00×)	4925.280 (0.17×)	990.102 (0.85×)	976.318 (0.86×)	287.469 (2.93×)
2^{14}	2^{15}	1647.855 (1.00×)	9678.936 (0.17×)	1803.966 (0.91×)	2005.452 (0.82×)	396.057 (4.16×)

Cuadro 2: Tiempos totales de ejecución en ms para las distintas implementaciones y, entre paréntesis, el *speedup* respecto a **Cublas** (tiempo_cublas/tiempo_version). El tiempo incluye la transferencia de datos Host→Device, la ejecución de todos los kernels necesarios para calcular la matriz de distancias y la transferencia Device→Host, medido mediante **cudaEvent**.

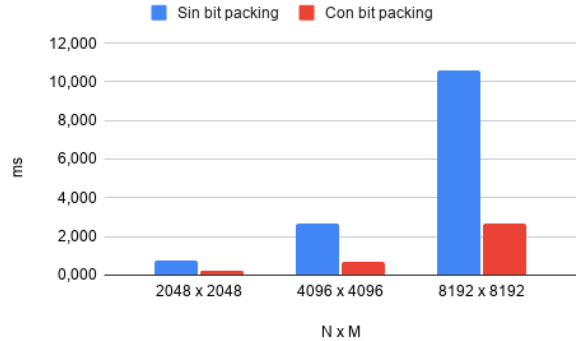
4.3. Tiempos de transferencia

Un punto al cual prestamos especial atención es la transferencia de la matriz de entrada X desde CPU a GPU, pues es uno de los mayores bottlenecks en programación en GPUs.

Para eso, usando **cudaEvents** medimos, en un promedio de 10 ejecuciones, este tiempo de transferencia para el uso con bit packing y lo comparamos con las versiones que no hacen bit packing.

N	M	Sin bit packing	Con bit packing
2048	2048	0.714	0.216
4096	4096	2.692	0.709
8192	8192	10.557	2.687

(a) Tiempos de transferencia.



(b) Comparación gráfica.

Figura 2: Comparación de los tiempos de transferencia Host-to-Device de la matriz de entrada. En la tabla (a) se ven los valores numéricos, que graficamos en (b).

Comparando, corroboramos que la versión con empaquetado de bits obtiene tiempos de transferencia inferiores. Esto es razonable, pues por lo ya explicado en la sección 3.2, ocupando menos memoria logramos representar la misma entrada. En la tabla 3 del Anexo se incluyeron los tiempos de transferencia para entradas más grandes y este comportamiento es consistente.

5. Análisis y Conclusiones

A lo largo del laboratorio comprobamos que reformular la distancia euclidiana fue un paso fundamental, ya que nos permitió tratar el problema central como una multiplicación de matrices

(XX^T) . Al evaluar el desempeño de las distintas versiones, quedó en evidencia que nuestra principal limitante no era la capacidad de cómputo de la GPU, sino el ancho de banda de la memoria (es decir, el problema es fuertemente “memory bound”).

Esto explica por qué la versión que solo utiliza Tensor Cores en precisión *half* no logró superar a la referencia de cuBLAS. Si bien los Tensor Cores aceleran el cálculo aritmético, los datos se seguían transfiriendo a los multiprocesadores ocupando 2 bytes por SNP, lo que terminaba saturando el bus de memoria y limitando el rendimiento general.

La mejora más significativa la obtuvimos al optimizar la cantidad de lecturas a memoria global mediante la estrategia de empaquetado de bits (*Bit packing*). Al compactar la información y guardar 16 SNPs en una sola palabra, logramos que cada lectura trajera mucha más información útil a la memoria compartida. Este beneficio se nota especialmente al aumentar la dimensión M (cantidad de SNPs): cuanto más larga es la secuencia, mayor es el impacto de reducir las lecturas.

Al juntar ambas ideas en la versión final (*Bit packing + Tensor Cores*), logramos mitigar la limitación de la memoria y, al mismo tiempo, aprovechar el paralelismo masivo de los Tensor Cores binarios. Esta variante resultó ser la mejor de todas, alcanzando un *speedup* de hasta $5.64\times$ frente a cuBLAS en el escenario más favorable ($N = 2^{12}, M = 2^{17}$).

Por último, notamos un efecto importante al escalar la dimensión N frente a M . Este comportamiento se hace evidente al invertir sus valores en las pruebas (por ejemplo, comparar el caso $N = 2^{13}, M = 2^{14}$ contra $N = 2^{14}, M = 2^{13}$ en la gráfica 1). Como el kernel de multiplicación aprovecha la simetría y calcula únicamente el triángulo superior, la cantidad de operaciones realizadas es proporcional a $N^2 \times M/2$. Al invertir las dimensiones de forma que N sea el doble que M , el tiempo de cálculo de la multiplicación aumenta exactamente al doble (comprobado en los resultados de todas las variantes). Sin embargo, el tiempo total de ejecución del programa crece en una proporción mucho mayor (se cuadruplica). Esto ocurre porque las demás etapas del pipeline (como el ensamblado de la matriz de distancias, la raíz cuadrada y, fundamentalmente, la transferencia de la matriz de $N \times N$ de regreso a Host) tienen una complejidad espacial y temporal estricta de $O(N^2)$. Al duplicarse N , estas operaciones periféricas cuadruplican su carga de trabajo. Como estos pasos son compartidos por todas las implementaciones, terminan afectando el tiempo total, demostrando la importancia de analizar el problema como un flujo de datos completo y no solo desde el rendimiento aislado del kernel dominante.

Anexo

Acerca de las ejecuciones y pruebas

N	M	Cublas	Baseline CUDA	Bit packing	Tensor cores	Bit packing + Tensor cores
2^{11}	2^{13}	2.690	2.687	0.709	2.687	0.758
2^{12}	2^{13}	5.313	5.312	1.376	5.317	1.446
2^{13}	2^{13}	10.556	10.560	2.700	10.542	2.765
2^{14}	2^{13}	21.026	21.062	5.321	21.078	5.399
2^{12}	2^{12}	2.692	2.699	0.710	2.690	0.766
2^{12}	2^{14}	10.550	10.574	2.692	10.554	2.768
2^{12}	2^{16}	41.990	41.992	10.555	42.053	10.616
2^{12}	2^{17}	83.856	83.944	21.023	84.164	21.117
2^{13}	2^{14}	21.040	21.046	5.316	21.053	5.390
$3 \cdot 2^{12}$	2^{14}	31.499	31.535	7.933	31.507	8.024
2^{14}	2^{14}	41.969	41.977	10.553	42.138	10.636
2^{14}	2^{15}	83.902	83.956	21.040	84.018	21.109

Cuadro 3: Tiempo de la transferencia Host→Device (**H→D**) en ms para las distintas implementaciones.

N	M	Cublas	Baseline CUDA	Bit packing	Tensor cores	Bit packing + Tensor cores
2^{11}	2^{13}	1.658	1.600	0.116	1.657	0.119
2^{12}	2^{13}	1.603	1.587	0.117	1.612	0.118
2^{13}	2^{13}	1.588	1.582	0.117	1.592	0.115
2^{14}	2^{13}	3.084	3.083	0.204	3.083	0.204
2^{12}	2^{12}	0.819	0.801	0.063	0.818	0.063
2^{12}	2^{14}	3.192	3.159	0.221	3.177	0.225
2^{12}	2^{16}	12.585	12.633	0.859	12.615	0.856
2^{12}	2^{17}	25.145	25.252	1.757	25.203	1.684
2^{13}	2^{14}	3.158	3.158	0.226	3.159	0.220
$3 \cdot 2^{12}$	2^{14}	6.157	6.156	0.395	6.155	0.398
2^{14}	2^{14}	6.157	6.159	0.397	6.157	0.398
2^{14}	2^{15}	12.305	12.305	0.790	12.386	0.786

Cuadro 4: Tiempo del cálculo de normas (**Normas**) en ms para las distintas implementaciones.

N	M	Cublas	Baseline CUDA	Bit packing	Tensor cores	Bit packing + Tensor cores
2^{11}	2^{13}	3.675	38.027	6.597	4.254	0.811
2^{12}	2^{13}	21.862	148.477	25.564	16.262	3.201
2^{13}	2^{13}	56.144	589.472	100.849	63.604	12.798
2^{14}	2^{13}	253.538	2351.686	402.260	252.981	50.500
2^{12}	2^{12}	7.002	74.444	12.879	8.230	1.575
2^{12}	2^{14}	25.900	296.757	50.898	46.186	6.414
2^{12}	2^{16}	166.072	1188.249	202.719	278.525	25.163
2^{12}	2^{17}	352.404	2377.713	405.295	655.910	50.129
2^{13}	2^{14}	135.810	1178.531	201.400	187.408	25.209
$3 \cdot 2^{12}$	2^{14}	338.812	2646.816	452.675	423.650	56.534
2^{14}	2^{14}	617.604	4701.661	803.606	752.625	100.360
2^{14}	2^{15}	1380.794	9407.208	1607.221	1734.411	197.899

Cuadro 5: Tiempo del kernel de multiplicación XX^T (**Mult**) en ms para las distintas implementaciones.

N	M	Cublas	Baseline CUDA	Bit packing	Tensor cores	Bit packing + Tensor cores
2^{11}	2^{13}	0.101	0.099	0.102	0.101	0.109
2^{12}	2^{13}	0.403	0.402	0.404	0.404	0.440
2^{13}	2^{13}	1.646	1.645	1.649	1.647	1.783
2^{14}	2^{13}	6.590	6.596	6.599	6.599	7.148
2^{12}	2^{12}	0.406	0.403	0.407	0.406	0.439
2^{12}	2^{14}	0.404	0.401	0.403	0.402	0.438
2^{12}	2^{16}	0.401	0.401	0.401	0.401	0.433
2^{12}	2^{17}	0.400	0.400	0.399	0.400	0.430
2^{13}	2^{14}	1.644	1.643	1.643	1.645	1.776
$3 \cdot 2^{12}$	2^{14}	3.691	3.696	3.696	3.696	3.999
2^{14}	2^{14}	6.599	6.579	6.591	6.588	7.137
2^{14}	2^{15}	6.597	6.597	6.600	6.597	7.139

Cuadro 6: Tiempo de la construcción de la matriz de distancias al cuadrado (**Dist**) en ms para las distintas implementaciones.

N	M	Cublas	Baseline CUDA	Bit packing	Tensor cores	Bit packing + Tensor cores
2^{11}	2^{13}	0.071	0.069	0.070	0.070	0.070
2^{12}	2^{13}	0.269	0.269	0.269	0.269	0.270
2^{13}	2^{13}	1.082	1.083	1.083	1.083	1.084
2^{14}	2^{13}	4.317	4.317	4.318	4.318	4.317
2^{12}	2^{12}	0.271	0.268	0.270	0.270	0.270
2^{12}	2^{14}	0.269	0.268	0.269	0.268	0.270
2^{12}	2^{16}	0.268	0.268	0.268	0.268	0.269
2^{12}	2^{17}	0.269	0.268	0.268	0.268	0.268
2^{13}	2^{14}	1.083	1.083	1.082	1.082	1.084
$3 \cdot 2^{12}$	2^{14}	2.443	2.443	2.442	2.440	2.443
2^{14}	2^{14}	4.317	4.317	4.318	4.316	4.318
2^{14}	2^{15}	4.315	4.318	4.318	4.318	4.318

Cuadro 7: Tiempo de la raíz cuadrada final (**Sqrt**) en ms para las distintas implementaciones.

Referencias

- [1] Facultad de Ingeniería. *Diapositivas del curso: Programación de Propósito General en GPUs (GPGPU)*. Facultad de Ingeniería, Universidad de la República (UdelaR), 2026.
- [2] NVIDIA Corporation. *CUDA C++ Best Practices Guide - Throughput of Native Arithmetic Instructions*. NVIDIA CUDA Toolkit Documentation, 2026. Disponible en: <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html#throughput-of-native-arithmetic-instructions>
- [3] NVIDIA Corporation. *CUDA C++ Programming Guide - Warp Matrix Functions, Sub-byte Operations*. NVIDIA CUDA Toolkit Documentation, 2026. Disponible en: <https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/cpp-language-extensions.html#sub-byte-operations>